

Design and Implementation of a Token-NFT-Liquidity Smart Contract Suite

1st Luca Sforza
sforza.2050030@studenti.uniroma1.it

2nd Roberto Di Rosa
dirosa.2043388@studenti.uniroma1.it,

Abstract—Smart contracts allow the creation of applications that run on the blockchain. We aim to present a suite of smart contracts designed to manage NFT (Non-Fungible Token) auctions using a custom token called SapiCoin. We have deployed the auction on the blockchain that utilizes SapiCoin, and to obtain SapiCoin in order to participate in the auction, we have created a Liquidity Pool that enables the exchange of Ether for SapiCoin.

```
contract SapiCoin is ERC20 {  
    /* implementation */  
}
```

Fig. 1. Polimorfism on Solidity

I. INTRODUCTION

From a programming perspective, Smart Contracts can be seen as classes in traditional programming languages such as Java.

To create a Token, one must therefore define a class that represents the Token and manages transactions involving that currency.

Other smart contracts (e.g., an auction) can use Tokens as a means of payment within the contract itself.

However, to avoid binding a smart contract to a specific Token and to promote code reusability, the concept of a Token can be abstracted.

The Ethereum community standardized this abstraction through a document known as ERC-20.

A Token is therefore represented by an Interface (a concept similar to those found in other programming languages).

A smart contract that aims to function as a Token must extend this interface and implement its methods.

The same concept is applied to NFTs, standardized through ERC-721.

In contrast, there is no community standard for auctions, so the design of such smart contracts must be created from scratch.

II. TOKEN

ERC-20 è lo standard che rappresenta un token nelle blockchain EVM compatibili.

Non esiste una figura centrale che possono emettere questi standard, ma community come Ethereum Magician , permettono di discutere di miglioramenti dello standard Ethereum tramite IEA

Lo standard impone che un token è uno smart contract con determinati metodi implementati.

A. Polimorfismo dell'EVM

Per fare questo si necessita del polimorfismo.

Nell'Ethereum Virtual Machine non esistono istruzioni per gestire classi astratte, ma la VM stessa è poliformica.

Quando si chiama un metodo di uno smart contract tramite una transazione, per accedere all'indirizzo di memoria del metodo si esegue l'hash della segnatura della funzione.

Nel caso il metodo chiamato non esiste allora viene chiamata una funzione di fallback di default (che può essere sovrascritta).

Grazie a questo funzionamento dell'EVM è possibile implementare un qualsiasi standard.

Se volessimo implementare un contratto che gestisce un asta di NFTs allora esso chiamerà le funzioni che hanno l'hash della segnatura come quelle definite dallo standard ERC-20.

B. Design of the standard ERC-20

Uno smart contract lo possiamo visualizzare come una classe.

Linguaggi di programmazione come solidity permettono di rappresentare l'idea di Interfaccia. Quindi possiamo vicolare uno smart contract ad avere i metodi imposti dallo standard ereditando i metodi astratti di un interfaccia come mostrato in Fig. 1.

Ovviamente questo è puro zucchero sintattico poiché la EVM stessa è già poliformica , infatti l'ereditarietà può essere omessa.

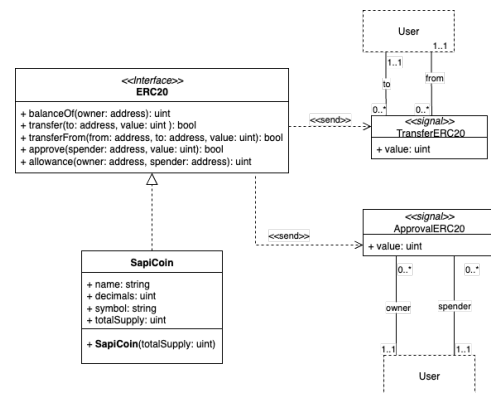


Fig. 2. UML Class Diagram of a Token.

```

1 event Transfer(address indexed _from, address
  indexed _to, uint256 _value);
2 event Approval(address indexed _owner, address
  indexed _spender, uint256 _value);

```

Fig. 3. Events on Solidity

In Figura 2 è possibile vedere un overview dello standard ERC-20.

Lo standard ERC-20 impone per un token di implementare i seguenti metodi:

- 1) **balanceOf(address owner)** ritorna la quantità di token che un certo address possiede
- 2) **transfer(address to, uint256 value)** permette di inviare dei token verso un determinato address (parametro `to`).
- 3) **approve(address spender, uint256 value)** permette ad un address (lo `spender`) di poter spendere una certa quantità di token che possiede l'address che invia la transazione.
- 4) **transferFrom(address from, address to, uint256 value)** permette di trasferire da un certo address (owner dei token) (non necessariamente quello che esegue la transazione) verso un altro address una determinata quantità di token specificata in `value`. Chiaramente non deve essere possibile a chi non ha diritto di spendere i nostri token, infatti chi esegue la transazione deve essere stato approvato (tramite il metodo `approve`) di poterla eseguire.
- 5) **allowance(address owner, address spender)** permette di sapere quanti token un `owner` ha approvato verso uno `spender`.

Tutto il sistema di approve è molto utile da un punto di vista programmatico.

Noi utenti possiamo chiamare approve verso un certo smart contract per una determinata quantità di token.

Cos'ì uno smart contract può usare i nostri token per offrire un servizio.

ERC-20 offre anche un insieme di eventi che un token dovrebbe emettere quando cambia lo stato interno del contratto, così le operazioni avvenute in uno smart contract possono essere rintracciate e visualizzate da applicazioni frontend come etherscan.

Nella figura 2 sono descritti i vari eventi emessi in modo asincrono dall'ERC-20, in particolare:

- 1) Transfer che traccia i trasferimenti di token tra utenti.
- 2) Approval che traccia le approvazioni da owner a spender.

In Solidity esiste un costrutto per rappresentare questi eventi come mostrato in Fig. 3.

Per implementare uno specifico Token, nel nostro caso SapiCoin, non è presente una logica extra oltre ad essere un token. Se volessimo implementare uno stablecoin oppure un wrapped coin dovremmo aggiungere metodi per gestire lo scambio del token con la vera valuta che rappresenta (per esempio USDT o WETH).

L'implementazione delle altre funzionalità standard di un token è facilmente reperibile online (in alcuni casi sono embedded in ambienti di sviluppo solidity).

L'unica cosa modificata sono le vanità del token, ovvero il nome del token, la quantità di decimali che il token possiede (nel nostro caso 18 decimali) e il simbolo del token (SAC).

Avendo messo 18 decimali la quantità più piccola di SapiCoin che è possibile ottenere è $1 \cdot 10^{-18}$.

Però essendo solo una vanità, a basso livello abbiamo accesso solo ad un numero intero unsigned da 256 bit.

Quindi ad esempio 4 SapiCoin in memoria è salvato come $4 \cdot 10^{18}$.

III. NFTs

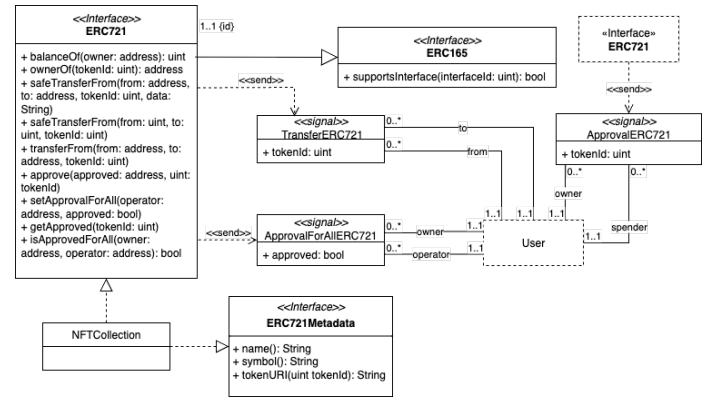


Fig. 4. UML Class Diagram of a NFT.

Secondo lo standard very ERC-721 compliant contract must implement the ERC721 and ERC165 interfaces come mostrato in figura 4.

A. Standard ERC-165

Lo standard ERC-165 definisce un meccanismo per dichiarare e verificare se un contratto implementa una determinata interfaccia. In altre parole, serve per permettere ad altri contratti di interrogare un contratto e capire se supporta una certa funzionalità (ad esempio ERC-721, ERC-20, ecc.).

supportsInterface(bytes4 interfaceID): permette di verificare se il contratto implementa l'interfaccia identificata da `interfaceID`. Restituisce `true` se l'interfaccia è supportata e `false` altrimenti.

B. Standard ERC-721

L'interfaccia ERC-721 definisce lo standard per i NFT. Ogni token è unico e distinto, a differenza degli ERC-20 che sono fungibili (tutti uguali). Le funzioni da implementare sono:

- 1) **balanceOf(address owner):** restituisce il numero di NFT posseduti da un determinato indirizzo.
- 2) **ownerOf(uint256 tokenId):** restituisce l'indirizzo del proprietario del token identificato da `tokenId`.
- 3) **safeTransferFrom(address from, address to, uint256 tokenId, bytes data):** trasferisce in modo sicuro la proprietà di un NFT da un indirizzo a un altro. Se il

destinatario è un contratto, viene chiamata la funzione `onERC721Received` per verificare che possa ricevere NFT. Per gli scopi di questo progetto abbiamo ignorato questo caso e per il contratto `auction` non abbiamo implementato la funzione `onERC721Received`. Genera un errore se:

- `msg.sender` non è il proprietario o un operatore autorizzato;
- `from` non è l'attuale proprietario;
- `to` è lo zero address;
- il token non esiste.

- 4) **`safeTransferFrom(address from, address to, uint256 tokenId)`**: identica alla funzione precedente, ma senza il parametro aggiuntivo `data` (che viene impostato a stringa vuota).
- 5) **`transferFrom(address from, address to, uint256 tokenId)`**: trasferisce la proprietà di un NFT senza effettuare controlli aggiuntivi sulla capacità del destinatario di riceverlo. È responsabilità del chiamante assicurarsi che `to` sia in grado di gestire NFT. Genera errore nelle stesse condizioni di `safeTransferFrom`.
- 6) **`approve(address approved, uint256 tokenId)`**: consente al proprietario di approvare un altro indirizzo (`approved`) per trasferire il token specificato (`tokenId`). Se viene passato l'indirizzo zero, rimuove ogni approvazione esistente.
- 7) **`setApprovalForAll(address operator, bool approved)`**: permette a un utente di abilitare o disabilitare un operatore che può gestire tutti i suoi NFT. Genera l'evento `ApprovalForAll` e consente di avere più operatori per lo stesso proprietario.
- 8) **`getApproved(uint256 tokenId)`**: restituisce l'indirizzo approvato per la gestione di un determinato token. Se non esiste alcun indirizzo approvato, restituisce lo zero address. Genera errore se il token non è valido.
- 9) **`isApprovedForAll(address owner, address operator)`**: verifica se un operatore è stato approvato per gestire tutti i token di un determinato proprietario. Restituisce `true` se è autorizzato, `false` altrimenti.

Come ERC-20 i contratti che implementano ERC-721 dovrebbero emettere i seguenti eventi:

- 1) **`Transfer(address indexed from, address indexed to, uint256 indexed tokenId)`**: viene emesso ogni volta che la proprietà di un NFT cambia, sia in caso di trasferimento, creazione (`from == 0`) o distruzione (`to == 0`). Quando un trasferimento avviene, l'eventuale indirizzo approvato per quel token viene automaticamente azzerato.
- 2) **`Approval(address indexed owner, address indexed approved, uint256 indexed tokenId)`**: viene emesso quando viene impostato o rimosso un indirizzo approvato per un NFT specifico. L'indirizzo zero indica che non esiste più un indirizzo approvato per quel token.
- 3) **`ApprovalForAll(address indexed owner, address indexed operator, bool approved)`**: viene emesso quando

un proprietario abilita o disabilita un operatore che può gestire tutti i suoi NFT. L'operatore può quindi effettuare trasferimenti o approvazioni per conto del proprietario.

C. Metadata e vanità opzionali

Estendere l'interfaccia `ERC721Metadata` non è obbligatorio per lo standard ERC-721, serve unicamente per aggiungere vanità ai NFT.

Grazie a questa interfaccia possiamo ottenere il nome della collezione NFT, il simbolo della collezione e tramite il metodo **`tokenURI(uint256 tokenId)`** possiamo ottenere URL associato all'NFT.

Questo link può essere verso l'immagine che NFT rappresenta o altre informazioni.

Per esempio Ferrari crea un NFT per una macchina venduta, quindi potrebbero implementare questa interfaccia per la loro collezione NFT per dare in output un link ad una pagina internet dove sono salvate tutte le informazioni della macchina.

Dipende molto dalle funzionalità che vogliamo incorporare nel nostro NFT, ma l'architettura dietro agli NFT rimane sempre la stessa.

Infatti anche per lo standard ERC-721 come per lo standard ERC-20 possiamo trovare tantissime implementazioni da riusare per la nostra collezione.

Per lo scopo di questo progetto abbiamo creato un NFT per l'immagine 5.



Fig. 5. Immagine associata all'NFT che abbiamo fatto il minting

IV. AUCTION